Book 4

What It Cannot Do

Honest limits, with the evidence

The most useful documentation any tool can give you is a clear account of where it stops. Everything here is traceable to something real: a probe run against a live Ableton session, a verification pass, or the shape of the architecture itself. Nothing in this book is a guess, and several entries describe actions that ship today and quietly do nothing.

In this book

- 30 documented limits across 6 categories
- Actions that ship but are confirmed inert — and what to do instead
- Limits that come from Ableton rather than from this app
- Why it cannot hear your music, and what to use instead
- The failure modes of the chat model itself
- Common misconceptions that are not actually limits

Evidence drawn from live probe runs against Ableton Live 12.4.15b3, the LOM truth table, and the v1.0.40 verification pass.

Read this one first

Most frustration with Genius comes from expecting one of the things in this book. Ten minutes here will save you an afternoon of trying to make an inert action work. The single most important entry is “It cannot hear your music” — it is the assumption people most often make without noticing.

Actions that are shipping but currently do nothing

These exist in the tool list and will accept your request without complaining, but the underlying Ableton properties they depend on are not present in Live 12.4. They were probed directly against a running session, not merely assumed missing. Until they are removed or rebuilt, treat them as no-ops.

live_set_clip_fades cannot set audio clip fades

Asking Genius to fade a clip in or out will appear to succeed and will change nothing. Live's Clip.fade_in / fade_out properties are absent from real clips in this version. Do fades by hand in Live, or use volume automation via live_add_shaped_automation, which does work.

Evidence: Confirmed absent from both an audio clip's and a MIDI clip's real info dump in a running session (v1.0.40 probe finding 4).

live_set_clip_follow_action and live_get_clip_follow_action do nothing

Follow actions — 'play this clip, then jump to that one 30% of the time' — cannot be set or read. If you want generative Session behaviour, set it up manually in Live's clip view; Genius cannot reach it.

Evidence: The follow_action_* properties are confirmed absent from real clips (v1.0.40 probe finding 4, same root cause as clip fades).

live_map_rack_magnitude works, but cannot rename the macro

The useful half succeeds: it finds Dry/Wet parameters across a Rack's chains and binds them to Macro 1. The cosmetic half fails silently — the macro will not be relabelled 'Magnitude', because RackDevice.rename_macro and macro_map don't exist on a real Rack. Rename it yourself in Live.

Evidence: Confirmed absent from a real RackDevice's info dump (v1.0.40 probe finding 5; an earlier false positive from a loose substring match was corrected to a word-boundary regex).

live_set_appointed_device does not stick

Appointing a device for hardware control-surface follow does not persist. The write is issued, Live accepts it, and the value reads back empty even half a second later. The tool now reports an honest failure rather than pretending it worked — which is the correct behaviour, but it still means you cannot do this.

Evidence: Confirmed directly and repeatedly in v1.0.40 (finding 5). Suspected but unproven cause: the property may only hold inside a genuine control-surface session. Listed as genuinely open.

Genius cannot browse Live's own browser tree

It cannot walk the Live browser the way you can in the GUI — no browsing Packs, Places or the Core Library by category. Sample discovery goes through your configured disk folder instead, via live_find_samples and live_library.

Evidence: live_app browser confirmed absent: the LOM call returns the bare sentinel 0 rather than a real object reference (v1.0.40 probe finding 6).

Limits that come from Ableton, not from this app

Live's scripting interface simply does not expose these. No amount of work on Live Actions can add them; they would need Ableton to change Live itself.

It cannot load third-party VST or AU plugins

live_create_device inserts native Ableton devices only, and only on Live 12.3 and later. Serum, Massive X, your favourite reverb — Genius cannot put any of them on a track. It CAN read and control them in detail once you have loaded them yourself, via live_read_device, live_inspect_plugin and live_update_device.

Evidence: Live's device-creation API accepts native device names only. Plugin loading is not exposed to the Live Object Model at all.

It cannot render, export or bounce audio

There is no 'export my track' action. Genius cannot produce a finished WAV or MP3 of your song. The closest available thing is live_record_audio, which captures Live's live output through Windows loopback in real time — useful for analysis, not a substitute for a proper export.

Evidence: Live's Export Audio/Video dialog is not scriptable through the LOM.

It cannot freeze, flatten or consolidate

No freezing tracks, no flattening frozen tracks, no consolidating a time selection into a new clip. These are all GUI-only operations in Live.

Evidence: Not present in the Live 12.3.6 LOM truth table (app/lom_surface.json).

It cannot save your project or manage project files from inside Live

Genius cannot press Save, Save As, or create a new Live Set. It can READ .als files that already exist on disk via the als_* actions, but it cannot make Live write one. Save your own work.

Evidence: File operations are not exposed through the LOM.

It cannot undo reliably past its own changes

live_undo presses Live's native Undo, which means it rolls back whatever is most recent — including your own manual edits. And because one Genius action can expand into several Live undo steps, 'undo that' may need repeating.

Evidence: Inherent to using Live's own undo stack rather than a private one.

Two Song properties won't confirm a write within the same call

Song.loop and Song.session_automation_record do not reflect a change in their own immediate read-back. The tools now trust the write for these two specifically rather than failing a check that can be wrong. Everything else verifies correctly and synchronously.

Evidence: Proven directly in v1.0.40 (finding 3): the same set, done as a separate round-trip with a 50 ms gap, reads back correctly. Scoped to exactly these two properties — not applied as a blanket 'stop verifying' change.

It cannot hear your music

This is the most important limit in the whole guide, and the one people most often forget. Genius reasons about structure — notes, devices, parameter values, track layout. It has no ears in the ordinary sense.

It does not know what your track sounds like

It knows a Compressor is on track 2 with a 4:1 ratio. It does not know whether that sounds good, whether the vocal is sitting right, or whether the mix feels exciting. Asking 'does this sound good?' gets you a structural opinion dressed up as an aesthetic one.

Evidence: Architectural: the bridge transports JSON state, not audio.

Playback is not listening

live_playback starts Live's transport. Genius does not receive the audio. If you ask it to 'play this and tell me what you think', it will describe what is on the timeline, which is not the same thing.

Evidence: Architectural.

Real analysis requires a real-time recording first

The only route from your audio to a genuine measurement is live_record_audio or live_record_and_analyze, which capture Live's output through Windows loopback in real time. Ten seconds of audio takes ten seconds. Then live_analyze_audio gives you actual LUFS, true peak, crest factor and spectrum.

Evidence: By design — these are the only actions that touch real audio.

live_analyze_mix is structural, not acoustic

It is a heuristic audit of your Set's layout: it notices two tracks both occupying the low end, a missing limiter, lopsided panning. It reaches those conclusions by reading your project, not by listening to it. Useful, but do not mistake it for a mastering engineer's opinion.

Evidence: Implemented as structural reasoning over session state (app/mix_checker.py).

live_read_meter is one instant, not a measurement

It samples the meter at the moment you ask. It is not peak-hold, not integrated loudness, and not a running measurement. For anything that matters, record and analyse instead.

Evidence: Single LOM read of the current meter value.

It cannot act on its own

Genius is strictly request-and-response. There is no background process watching your session.

It cannot monitor or react to anything while you are not talking to it

It cannot watch for you to finish a take, notice that you have gone over budget on CPU, or act when playback reaches bar 33. Every action happens inside a turn you started.

Evidence: Architectural: tool calls only occur within a chat turn.

It cannot schedule anything for later

There is no 'do this in ten minutes' or 'do this every time I open the project'.

Evidence: Architectural.

It cannot respond to MIDI or controller input

It cannot be mapped to a knob, triggered by a pad, or driven from your Launchkey. It also cannot set up MIDI mappings for you — the LOM does not expose the mapping system.

Evidence: Not present in the LOM truth table.

Recording actions block for their full duration

A 30-second capture occupies 30 seconds. live_record_to_arrangement is worse: recording 32 bars takes 32 bars of real time. Nothing else happens while it runs.

Evidence: Inherent to real-time capture.

Limits of the chat model itself

Genius is an AI assistant driving a tool belt. The tool belt is deterministic; the driver is not. These are the failure modes to watch for.

It can claim to have done something it did not do

The single most common real-world failure. A model can narrate a convincing summary of changes it never made, especially at the end of a long multi-step job. Defend against it: ask for a live_session_diff, or re-read the thing it claims to have changed. Believe the read, not the summary.

Evidence: General LLM behaviour. This is why live_session_snapshot and live_session_diff exist as a pair.

It can guess parameter values instead of checking them

Without being asked to check, a model may assume a filter cutoff runs 0-1 when it actually runs 0-127, and write something meaningless. live_read_param_range was added in v1.0.39 specifically to stop this — but it has to actually be called. Ask for it explicitly on unfamiliar parameters.

Evidence: The motivating problem behind the v1.0.39 parameter-introspection work.

It can misread track numbering

Track numbers are positions as Live displays them. A track NAMED 'Track 3' may sit in position 7. Digits in a name mean nothing. Every action's schema warns about this, and it still trips models up — say 'the track called X' when you can, or confirm with live_get_selected.

Evidence: Explicit warnings in the tool schemas; the reason app/live_numbering.py and its test suite exist.

Quality varies significantly by model

Precise multi-step automation is genuinely harder than answering a theory question. A model that explains modal harmony beautifully may sequence six dependent tool calls badly. Book 3 tags every prompt with a suggested model for exactly this reason.

Evidence: Observed across the provider set supported in app/ai_providers.py.

Long autonomous builds drift

The more steps you ask for in one go, the more likely something in the middle goes sideways and everything after it compounds the error. Break big builds into checkpoints you can verify.

Evidence: General agentic behaviour; mitigated by the snapshot/diff pattern.

Things it is simply not for

Scope boundaries. None of these are bugs.

It is not a replacement for knowing Ableton

It is fastest and most accurate when you already know what you want and are using it to skip the tedious part. 'Rename all my default-named tracks' is a great use. 'Make me a hit record' is not a request it can evaluate.

Evidence: Scope.

It cannot install, update or configure Ableton or its packs

No installing Packs, no authorising Live, no managing your Ableton account, no changing Live's own preferences.

Evidence: Outside the LOM entirely.

It cannot work on a Set that is not open

Every live_* action needs Live running with the bridge device loaded. The exceptions are the offline families: als_* reads project files from disk, library_* reads the knowledge base, and the audio analysis actions work on files. Those work with Ableton closed.

Evidence: Architectural.

It cannot see your screen

It knows what is selected via live_get_selected, but it cannot see your layout, your plugin GUIs, or anything you have not asked it to read.

Evidence: Architectural.

live_transform_selected_notes is the least proven action here

It ships, and its companion read action verified correctly against a real session, but as of v1.0.40 it had never been exercised end to end with an actual non-empty note selection in the piano roll. Treat results with more suspicion than the rest of the toolset, and verify what it did.

Evidence: Listed as genuinely open in v1.0.40: the verification run returned a clean empty selection, so the transform path itself remains untested.

Things that are not actually limits

These come up often enough to be worth correcting directly.

The assumption	Actually
"It can't write real MIDI, it just describes music."	It writes actual notes into actual clips. live_write_notes and the whole generative family put real MIDI in your Set that you can then edit by hand.
"The generated music is the AI improvising."	The note data comes from deterministic algorithms — Bjorklund for Euclidean rhythms, Krumhansl-Schmuckler for key detection, real voice-leading rules for chords. The model chooses WHICH generator to call and with what settings; it does not invent the notes one at a time.
"It needs the internet to know music theory."	The knowledge base is local: a structured database of 42 curated datasets plus a folder of manuals on your own drive. No lookup leaves your machine.
"It can only do one thing per message."	A single prompt routinely expands into a dozen or more tool calls. The techno template prompt in Book 3 is one sentence and roughly twelve actions.
"If it says it did something, it did."	No. This is the one to stay sceptical about. Verify with a read or a diff when it matters.

Triage order when something breaks

Work down this list. It is ordered by how common the cause is, not how likely it is to be interesting.

#	Check	How	If that's it
1	**Is the bridge device in THIS Set?**	Look in Live for the Live Actions device.	Add it. This is the most common cause by a wide margin — it is per-project, not global.
2	**Is a dialog blocking Live?**	`live_read_dialog`	A stuck modal silently blocks every other action. Dismiss it and retry.
3	**Does the connection work at all?**	`live_connect`	If this fails, nothing downstream can work. Check Live is running and the ports match.
4	**Is Live overloaded?**	`live_read_cpu`	A struggling audio engine makes calls time out. Freeze or remove something.
5	**Is the action one of the inert ones?**	Check the first section of this book.	It will appear to succeed and change nothing. Use the documented alternative.
6	**Did it actually do what it claimed?**	`live_session_diff` or just re-read it.	If the diff is empty, it did not. Believe the read, not the summary.